CODESEEKER

LEE EUN CHONG

CODESEEKER

Toward the fix, hunting down causes like a SEEKER.

- I am not afraid to try new approaches to solve a problem.
- When a server stops, I trace the cause to the end with logs and measurements and stop it from recurring.



CONTENTS

Tech Stack

See my
tech stack

Google Study Jam

GCP training and
Gemini · Cloud Run use

ARGUS

Policy layer and checks
against runaway AI agents

Server Manager

Building and running
7 department GPU servers

DRAWMIND

DB and backend of
a board you draw on

AEye

Speeding up a walking
guide app for the blind

EcoReport

AI server that judges
wasted power from one photo

NYPC

How I used AI and built
the learning structure
in an AI agent battle

CONTENTS

TalkCompass	MUSEMATE	GridPuzzle
AI agent that builds prompts from meetings	Flutter app linking people by music · Lead	Solving a BOJ 814-2 grid problem with a genetic algorithm

Tech Stack

- Technologies I use for backend development.

Java | Spring Boot | JPA

MySQL | PostgreSQL

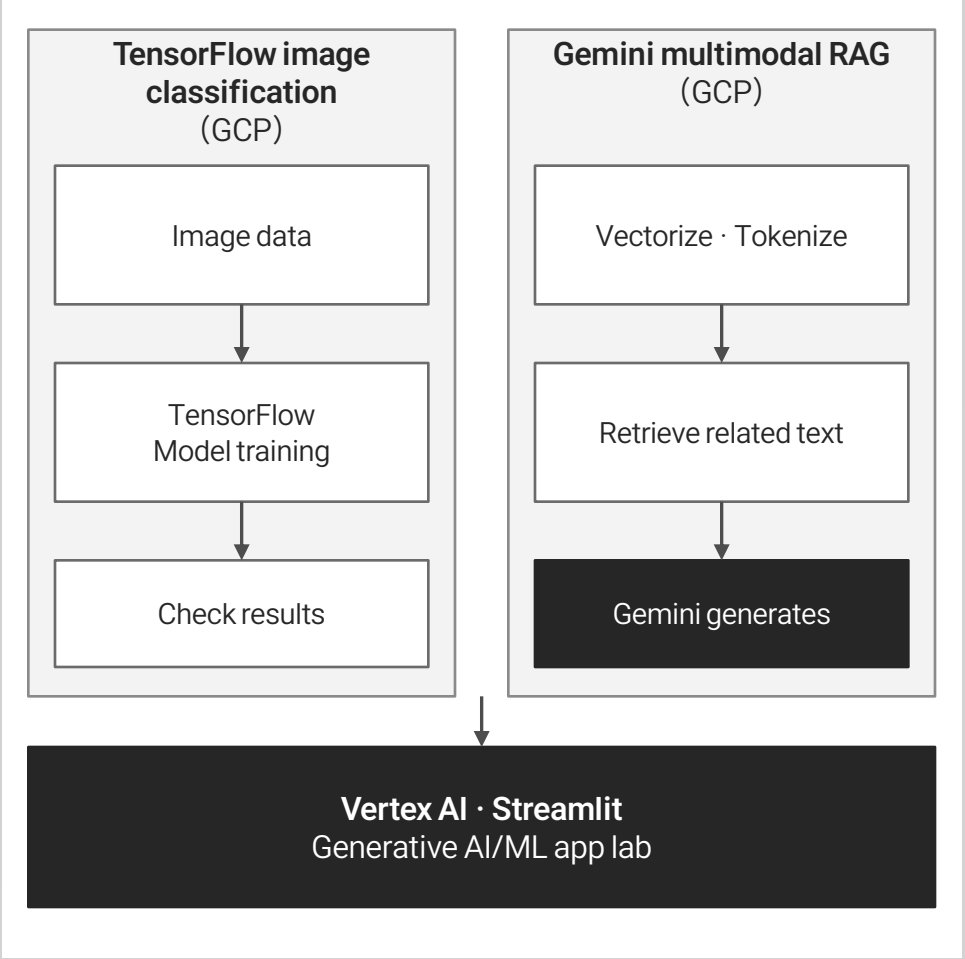
Docker | Linux | Kotlin | GCP

- Designed APIs and a DB with **Spring Boot**.
- Cut processing time with parallelism using **Kotlin coroutines**.
- Ran servers and traced failures with **Docker and Linux**.
- Completed Google Cloud Study Jam twice and deployed an AI server on Cloud Run.
- Built a tool-calling agent in TypeScript.

Google Study Jam

2024

- **Completed** the Google Cloud AI/ML Study Jam in **2024**.
- Practiced **TensorFlow image classification** on GCP.
- Analyzed documents with **multimodal RAG** and tried Vertex AI and Streamlit apps.

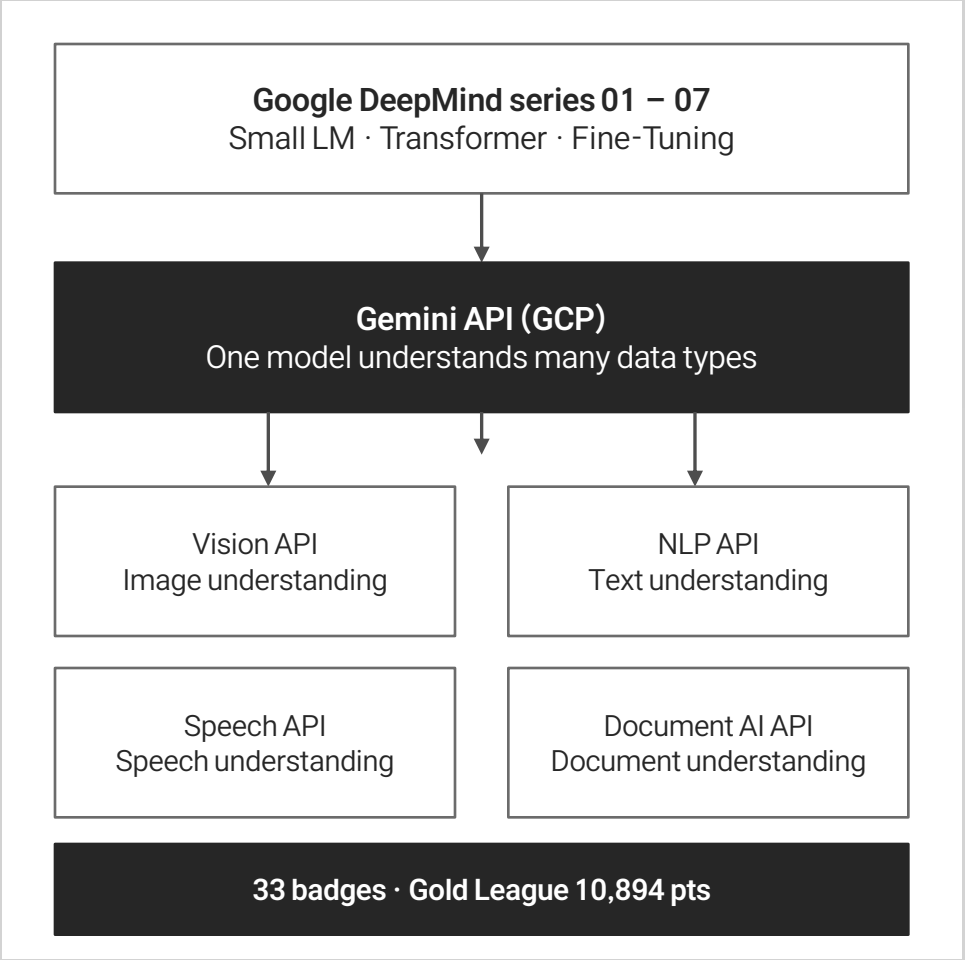


Links https://www.skills.google/public_profiles/d43aef75-3773-413c-a73c-fc5ad65a0df4

Google Study Jam

2026

- In **2026** I earned **33 badges** and **Gold League** (10,894 points) on Google Skills.
- Finished all **7 courses of the Google DeepMind series**.
- Practiced the **Vision, NLP, Speech, and Document AI APIs** and **multi-agent (ADK) deployment**.



Links

https://www.skills.google/public_profiles/c43db4d0-c4ea-454f-a306-db8fa38b46a3

Google Study Jam

Completion

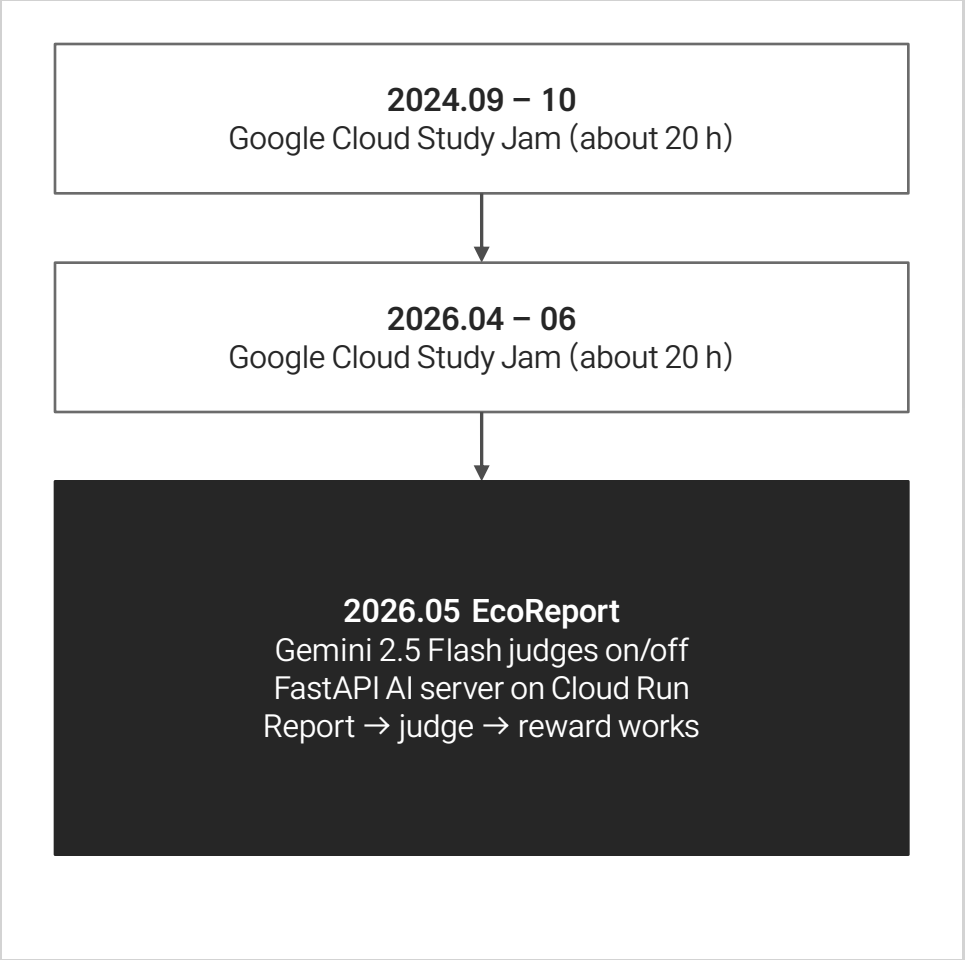
- This Google Cloud **bag** is the **completion swag**.
- Completed the program in **2024 and 2026**.
- The **bag photo** is attached as proof.



Google Study Jam

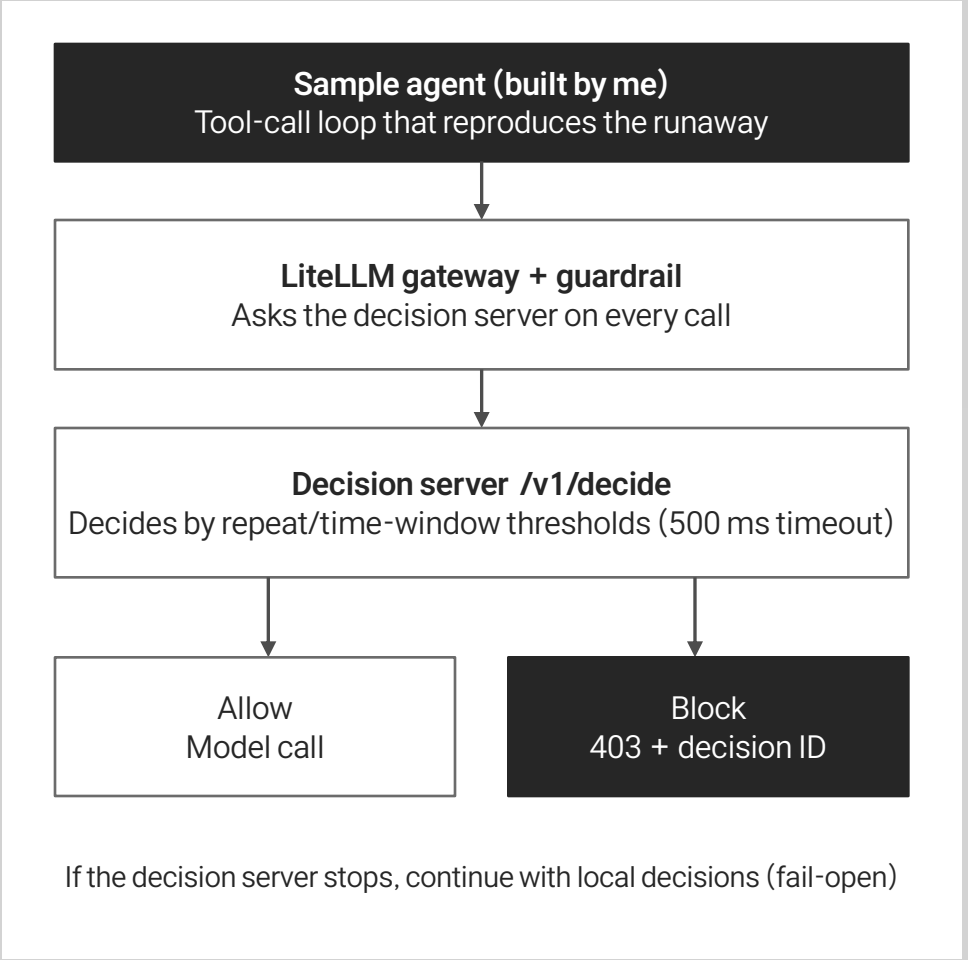
Applied to EcoReport

- Building on the Study Jam, EcoReport uses **Gemini 2.5 Flash** to judge on/off.
- Built the AI server with FastAPI and deployed it on **Cloud Run**.
- Runs **end to end** from report to point reward.



ARGUS
OVERVIEW

- A Kubernetes policy layer that stops **runaway repeated calls** from an agent before they reach the model.
- In a team of 4, I built the **sample agent** that reproduces the runaway, and a chat screen.
- I also ran the **real-data validation** of detection thresholds and decision latency.

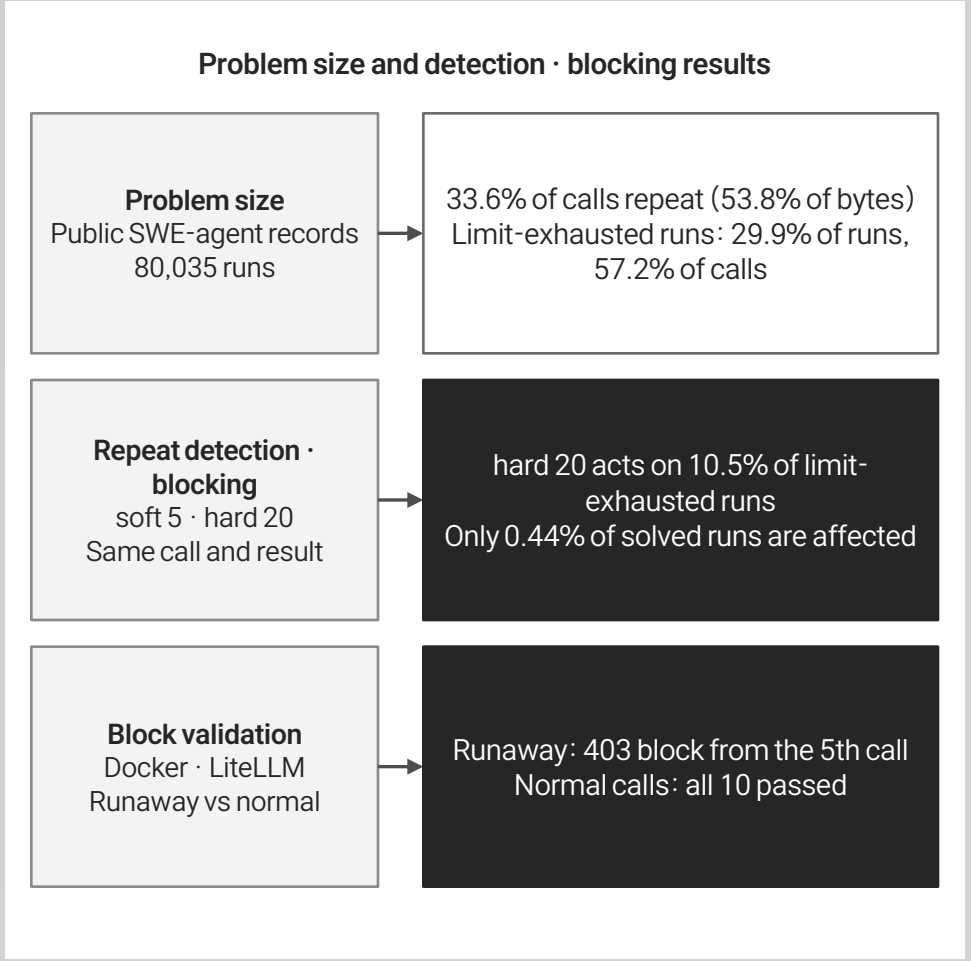


Links https://github.com/stp-argus/argus

ARGUS

Detection & Blocking

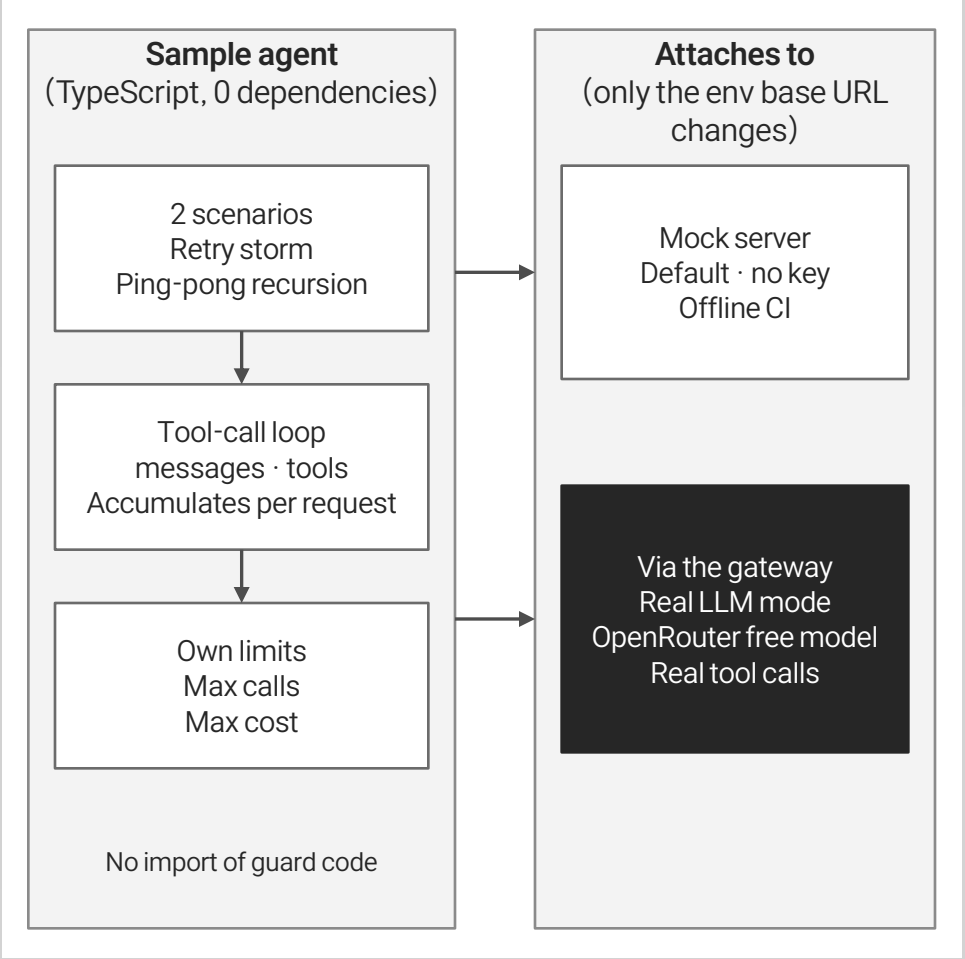
- In **80,000** public records, **33.6%** of calls were **repeats**.
- Repeats of the same call and result are **detected by fingerprint** and **blocked with 403** before reaching the model.
- The runaway is **blocked from the 5th call**, and **all 10** normal calls **passed**.



ARGUS

Sample Agent

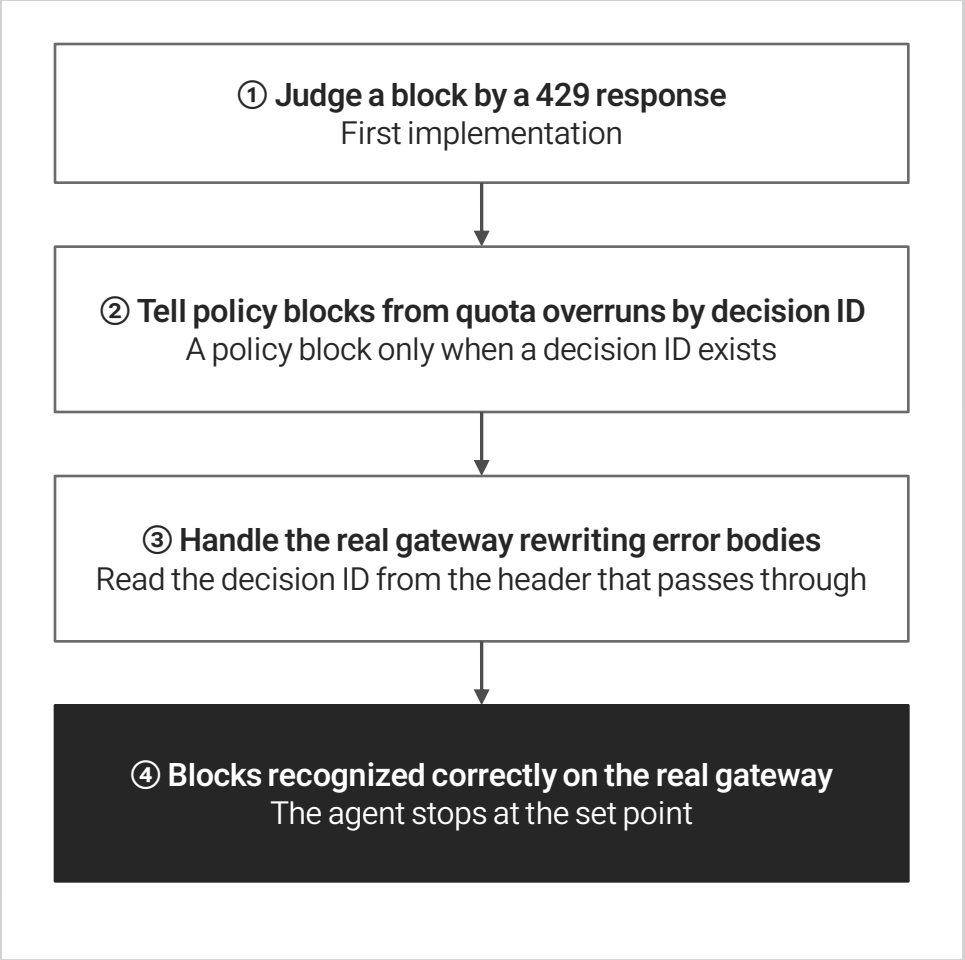
- An agent that reproduces retry storms and ping-pong recursion **the same way every time**.
- It knows nothing of the guard and attaches to the gateway by changing only the **base URL variable**.
- Mock by default, so it runs without a key, and it also supports a **real LLM mode**.



ARGUS

Block Detection

- Told policy blocks from quota overruns by **decision ID**.
- Reads the block from the **response header** even when the gateway rewrites the body.
- Blocks are **recognized as is** on a real gateway too.



ARGUS

Latency & Fail-open

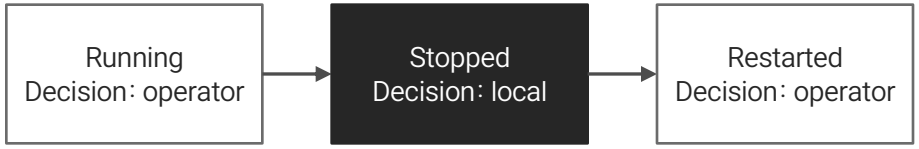
- The decision sits before every call, so I measured latency at **p50 · p99**.
- On a kind cluster (with auth), 1 concurrent request gave **p99 2.0–2.9 ms**.
- If the decision server stops, calls are not blocked and continue on a **local decision**.

Round-trip latency of one decision (ms)

Environment · concurrency	p50	p99
Local Docker · 1	1.6~2.6	3~7
kind cluster (auth) · 1	0.8~1.2	2.0~2.9
kind cluster · 32	20~38	75~153

Within the 500 ms timeout,
handled without errors

Log of stopping and restarting the decision server

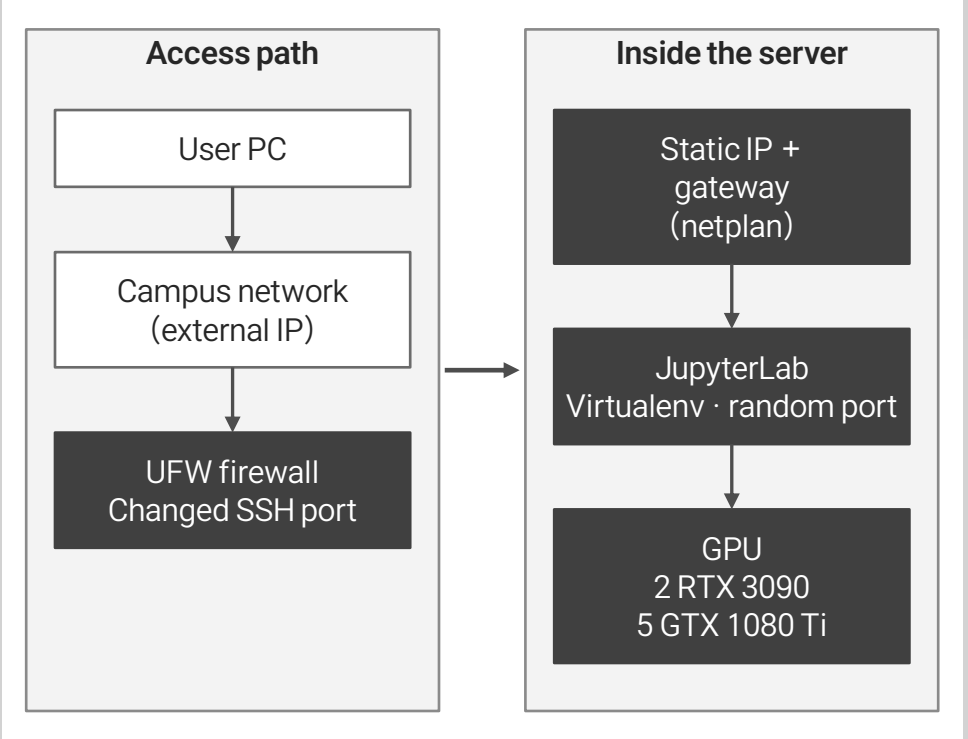


Calls still pass while it is stopped (fail-open),
and recover from the next call after restart

Server Manager

OVERVIEW

- Made **7** department GPU servers (2 RTX 3090, 5 GTX 1080 Ti) usable from outside.
- No one on campus knew Linux networking, so I **built and ran them alone**.
- After a server wipe I rebuilt them and **documented every step in a notebook**.



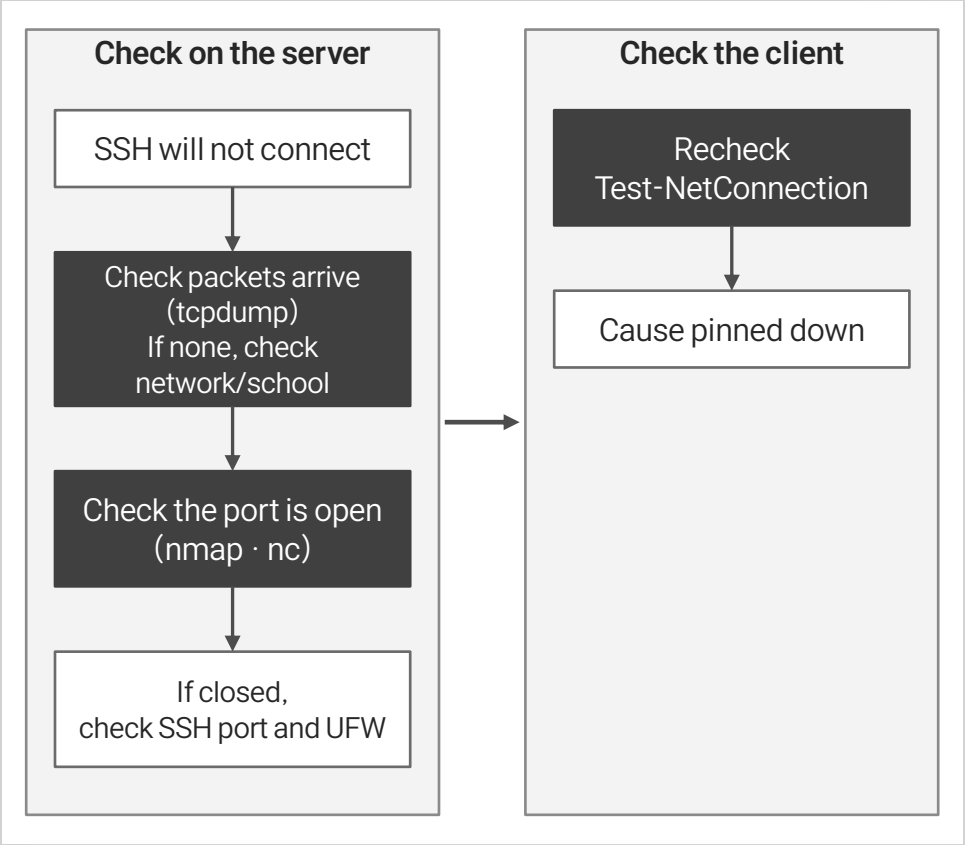
Links

https://github.com/BarrelZebra2149/ServerManagerProj

Server Manager

Server Network

- With a static IP, I first announced the gateway as **directly connected**, then set the default route.
- When SSH would not open, I narrowed the cause with **packet capture, port scan, and connection tests**.
- I also recorded the **network request step** for opening external ports.



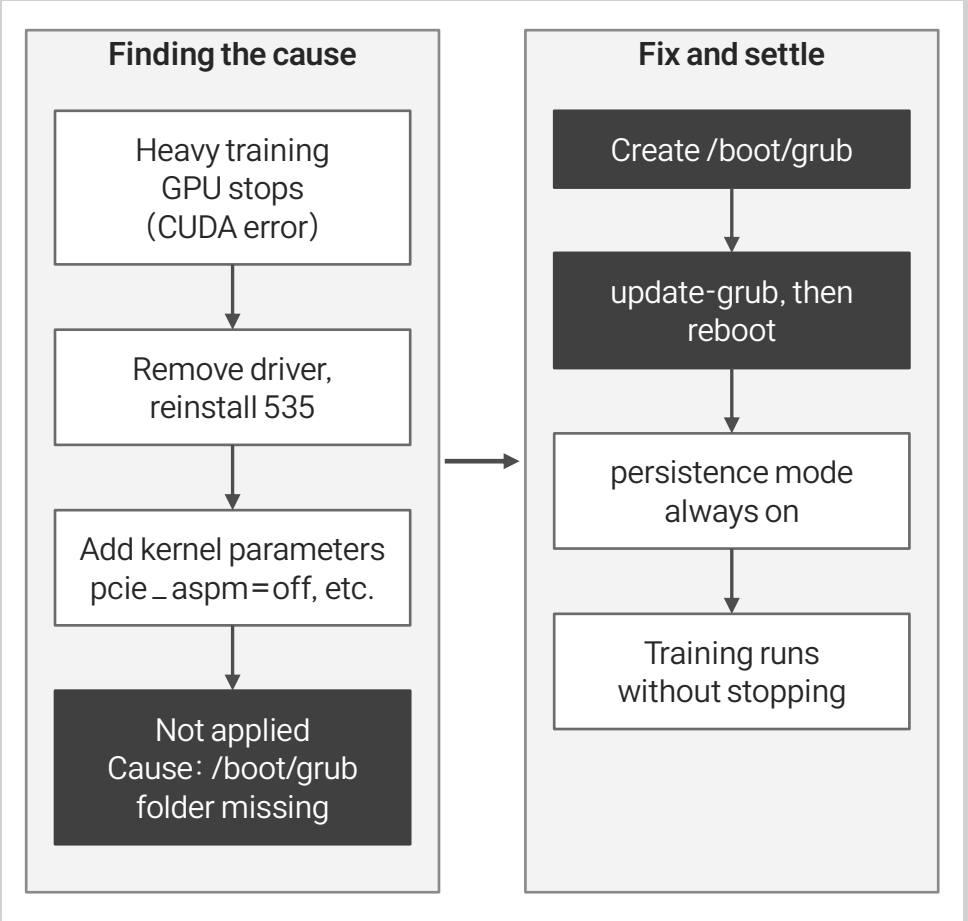
Links

https://github.com/BarrelZebra2149/ServerManagerProj

Server Manager

GPU problem Fix

- Heavy training kept **freezing the GPU** (CUDA error).
- I found kernel parameters were **not applied because /boot/grub was missing**, and fixed it.
- It **never recurred**, and I documented the fix.



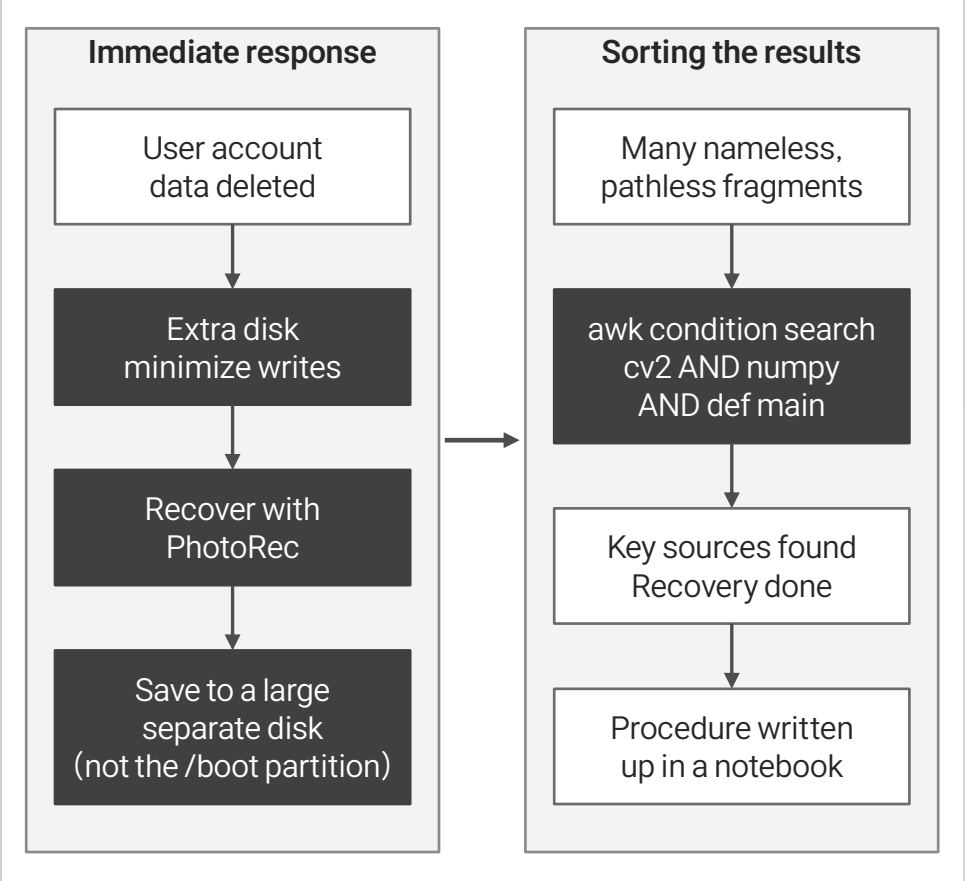
Server Manager

Data Recovery

- Recovered a deleted project.

  PhotoRec | awk | LVM

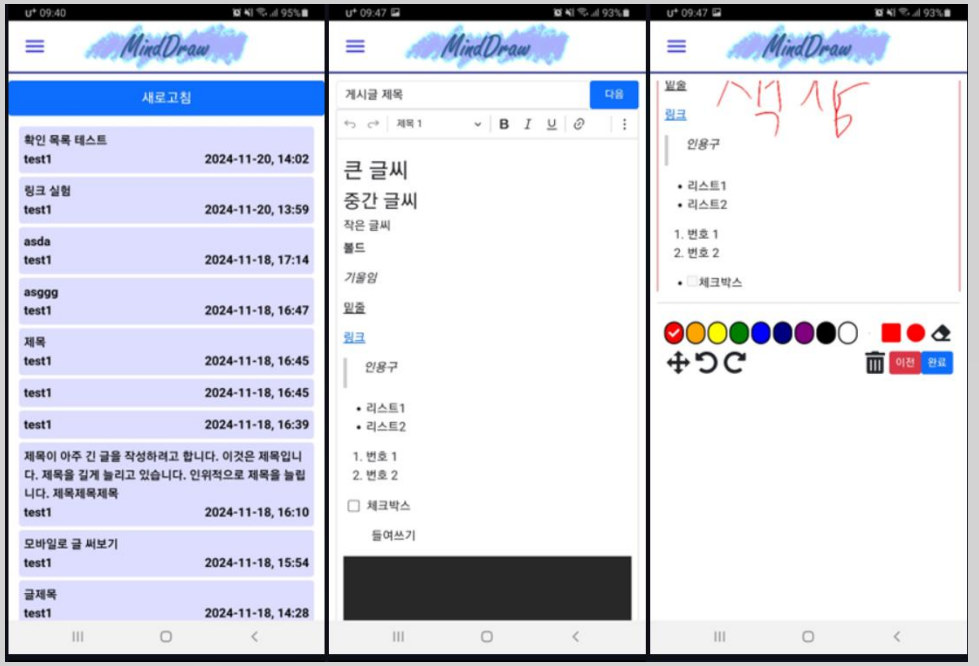
- Right after deletion I **minimized writes** and chose a **large separate disk** for recovery.
- I found the key sources among nameless fragments with an **awk condition search** (cv2 · numpy · main).



DRAWMIND

OVERVIEW

- A board where you **draw directly on text** and post it as it looks.
- As **team lead**, I wrote the plan and owned the **DB design and backend** (Spring Boot, PostgreSQL).



Links

https://github.com/Hoto-Mocha/TeamProject-DrawMind
https://uracle.co.kr/
https://www.morpheus.kr/

DRAWMIND

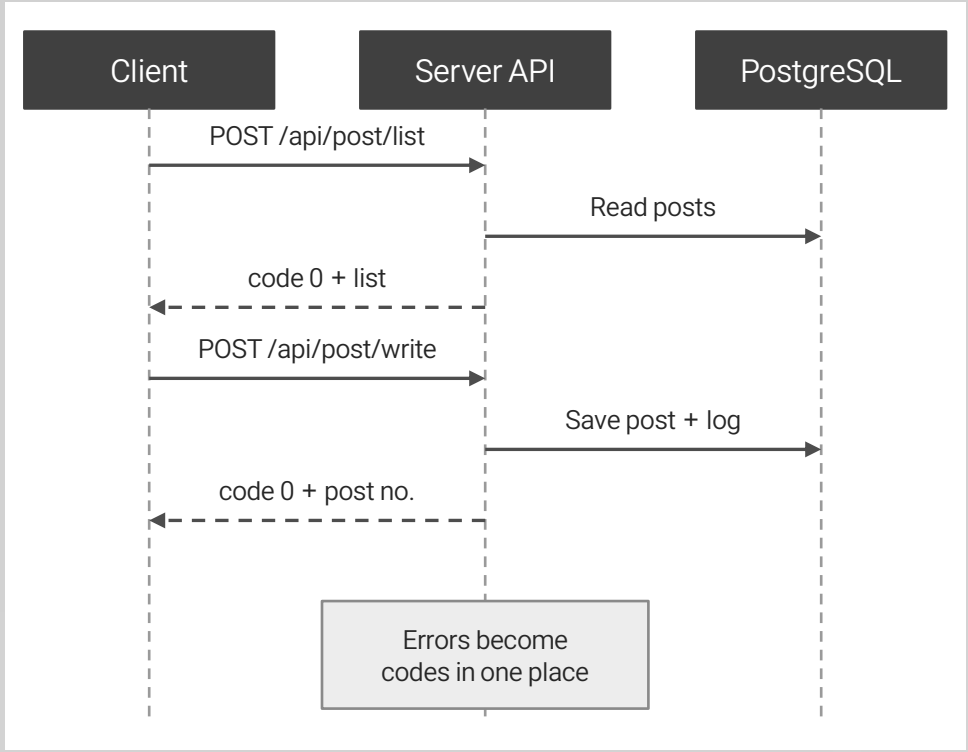
DRAWMIND API

- Before front end and back end split up, I wrote the **API spec** first.
- All 9 APIs are **POST · JSON**, and a **result-code table** unifies error responses.
- Layered into controller, service, and repository, errors are **converted to codes in one place**.

서비스명	게시판 리스트
URL	[host url]/api/post/list
Method	**POST**
Content-Type	**application/json**
설명	게시판 목록을 조회한다.

Request Params			
항목	타입	필수	설명
currentPage	Long	Y	페이지 일련번호 (1 일 경우 제일 처음부터)

Response (JSON)					
구분	항목	타입	필수	반복	설명
body	code	String	Y	N	결과 코드 (성공 : 0000)
	body	Array	N	N	목록
	postSeq	Long	Y	Y	일련번호
	postTitle	String	Y	Y	제목
	writer	String	Y	Y	작성자
	regDate	String	Y	Y	작성일시 (yyyyMMddHHmm)



Links

https://github.com/Hoto-Mocha/TeamProject-DrawMind

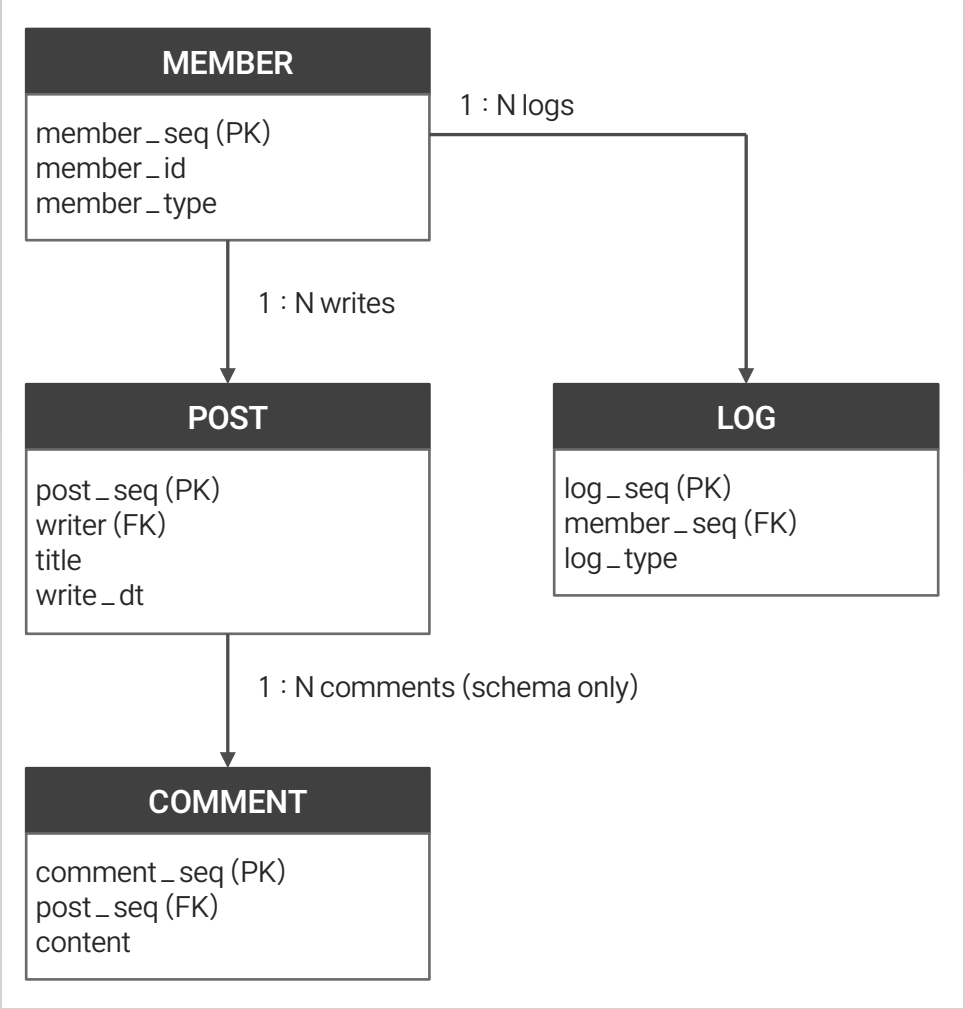
DRAWMIND

DRAWMIND Result

- Completed and integrated all 9 APIs.

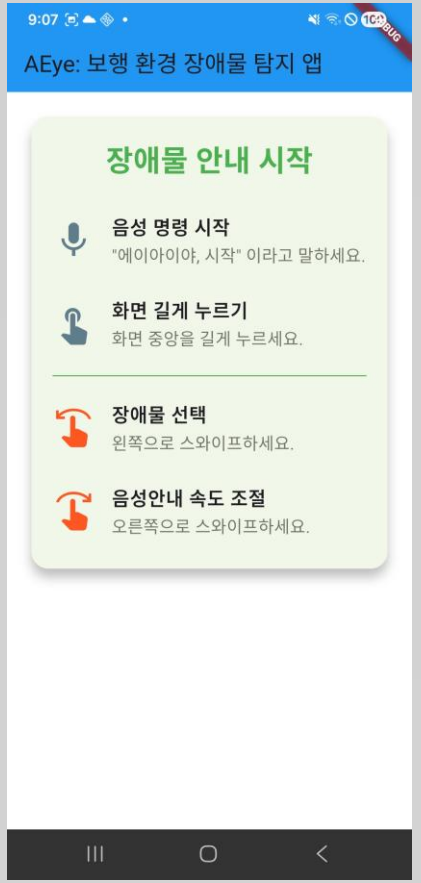
  Spring Boot | PostgreSQL

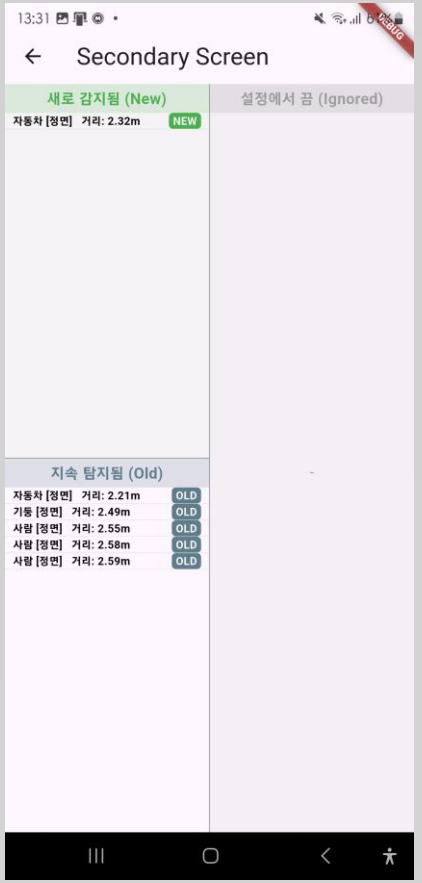
- Drafted a design with BCrypt password hashing and JWT authentication.



AEye
OVERVIEW

- An app that recognizes **obstacles, crosswalks, and roadways** on sidewalks and guides by voice.
- I owned **speeding up processing**: Depth model integration and a **parallel inference structure**.
- Before development, the whole team joined **interviews with visually impaired people** to confirm needed features.





Links

https://github.com/BumSooKim701/obstacle_detection_system_for_the_blind

AEye

AEye Result

- Running depth, detection, and segmentation as **parallel inference** cut the time per frame.
- Sequential sum **about 3.6 s → 830 ms average**
- Distance error (0–5 m) **1.70 m → 0.47 m** (team result)



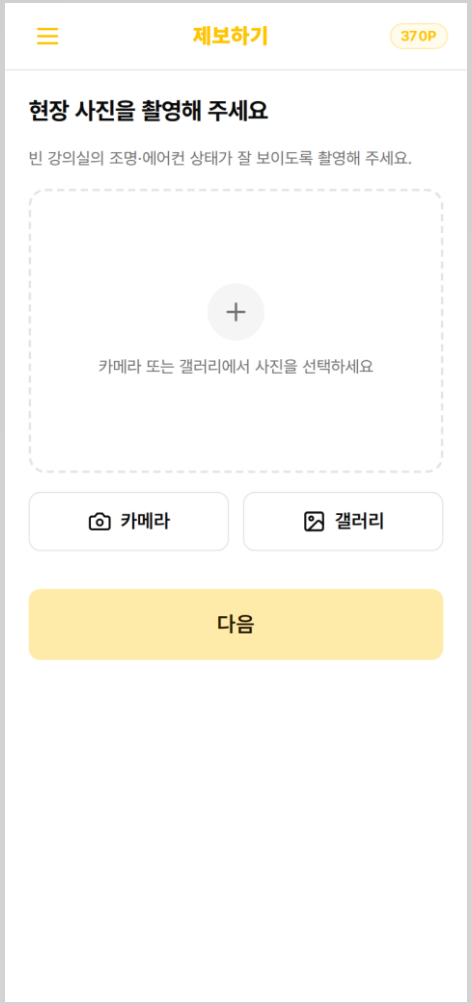
Links

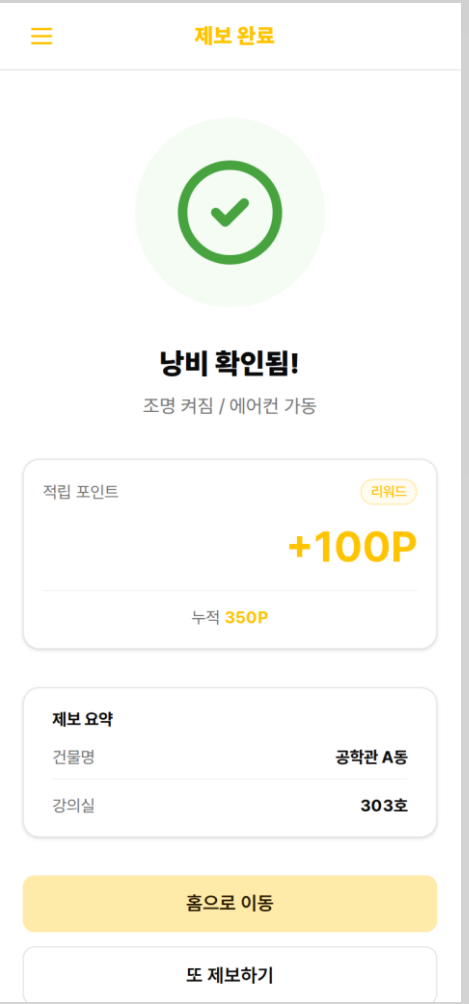
https://github.com/BumSooKim701/obstacle_detection_system_for_the_blind

EcoReport

OVERVIEW

- Report lights and air conditioners left on in an empty classroom with **one photo**, and AI judges the waste and savings.
- I owned the **AI judging server and backend integration**. (2-week hackathon entry)





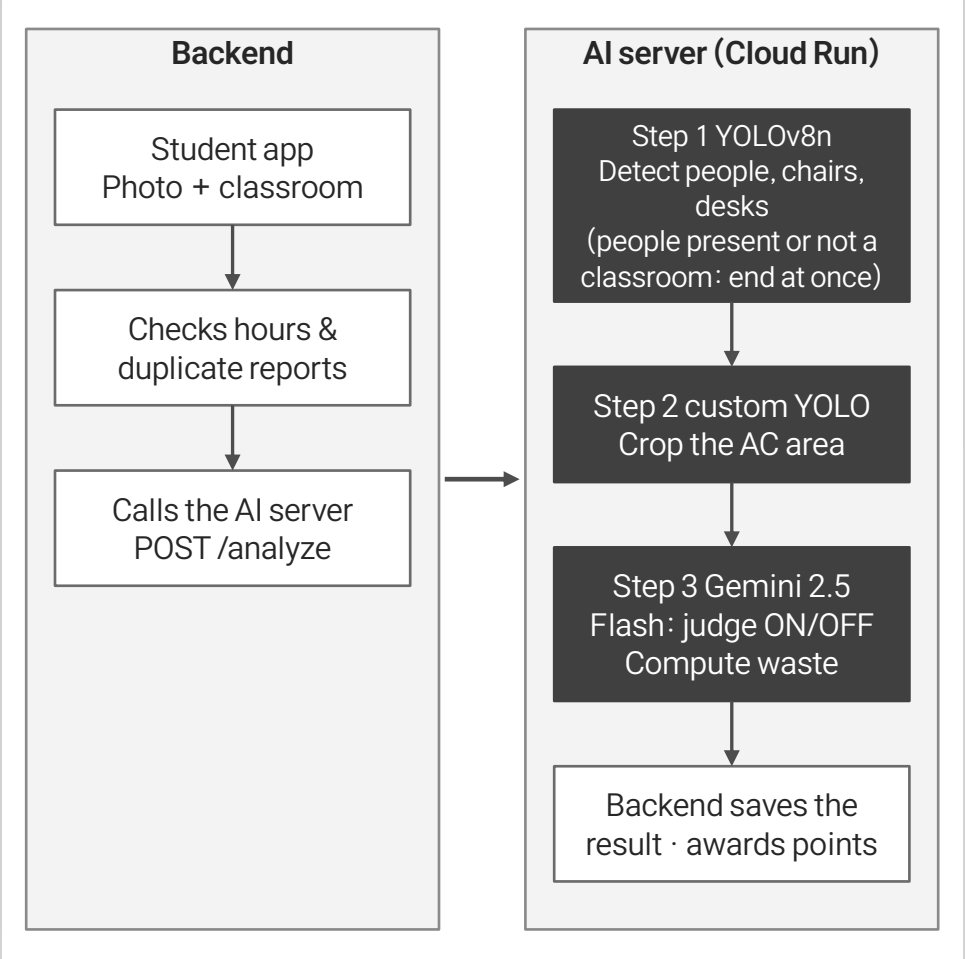
Links

https://github.com/BarrelZebra2149/duyuthon14_EcoReport

EcoReport

EcoReport AI Flow

- Step 1 **YOLOv8n**: if people are present or it is not a classroom, end without calling Gemini.
- Step 2 **custom YOLO**: trained by me, crops the air conditioner area.
- Step 3 **Gemini 2.5 Flash**: judges ON/OFF and computes the wasted amount.



Links

https://github.com/BarrelZebra2149/duyuthon14_EcoReport

EcoReport

EcoReport Result

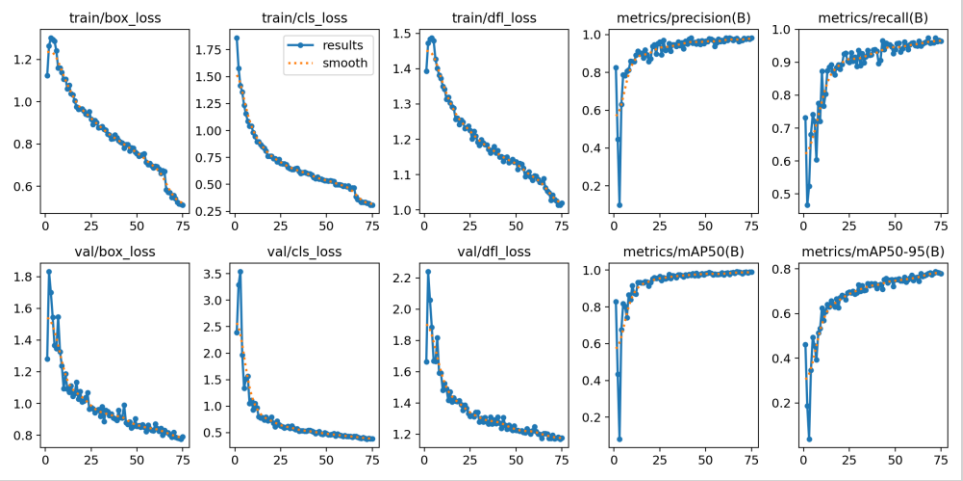
- Trained the air-conditioner detector on **2,016 images** and reached **mAP@0.5 of about 0.99**.
- The AI server runs on **Cloud Run**, so report → judge → points works.

Air-conditioner detector validation

mAP@0.5	Precision	Recall
0.989	0.983	0.964

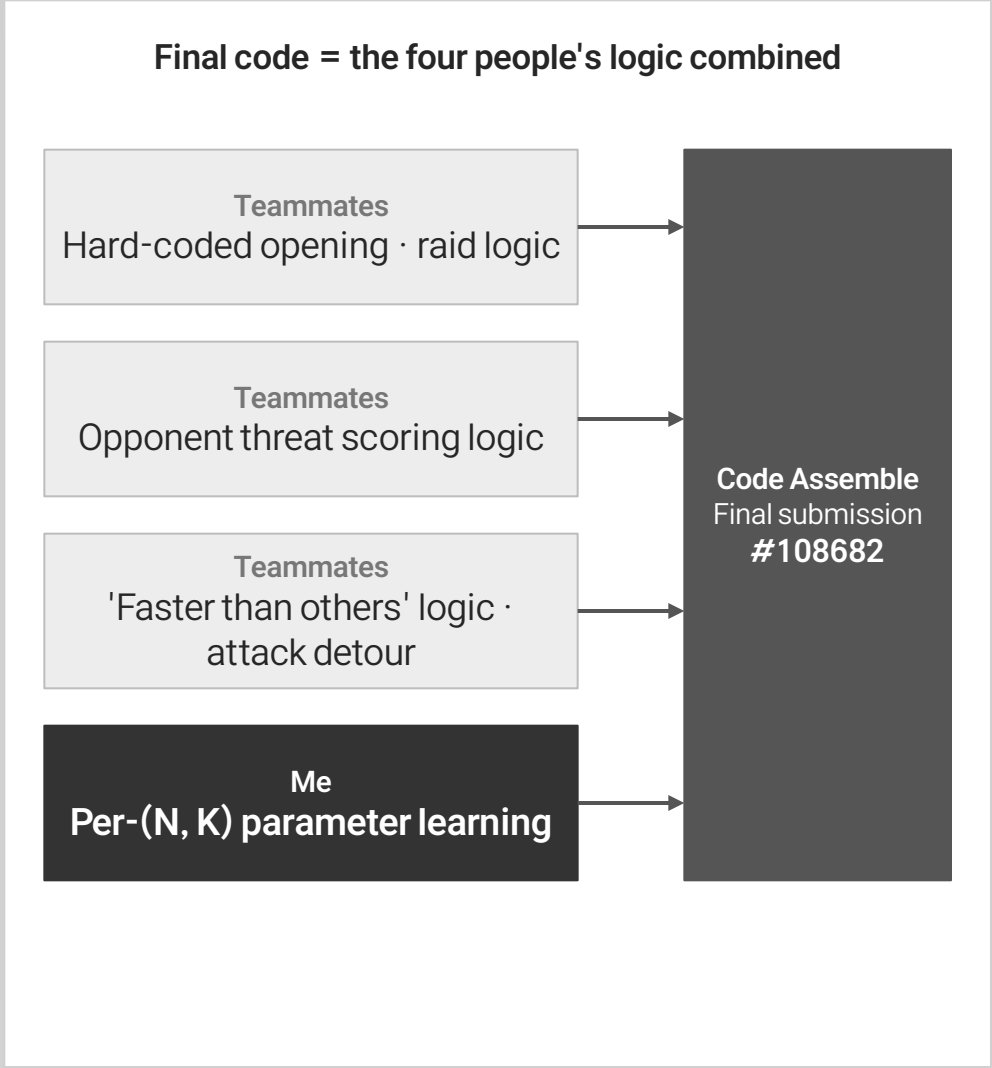
Trained on 2,016 public images · 75 epochs

Validation values at the last epoch



NYPC
OVERVIEW

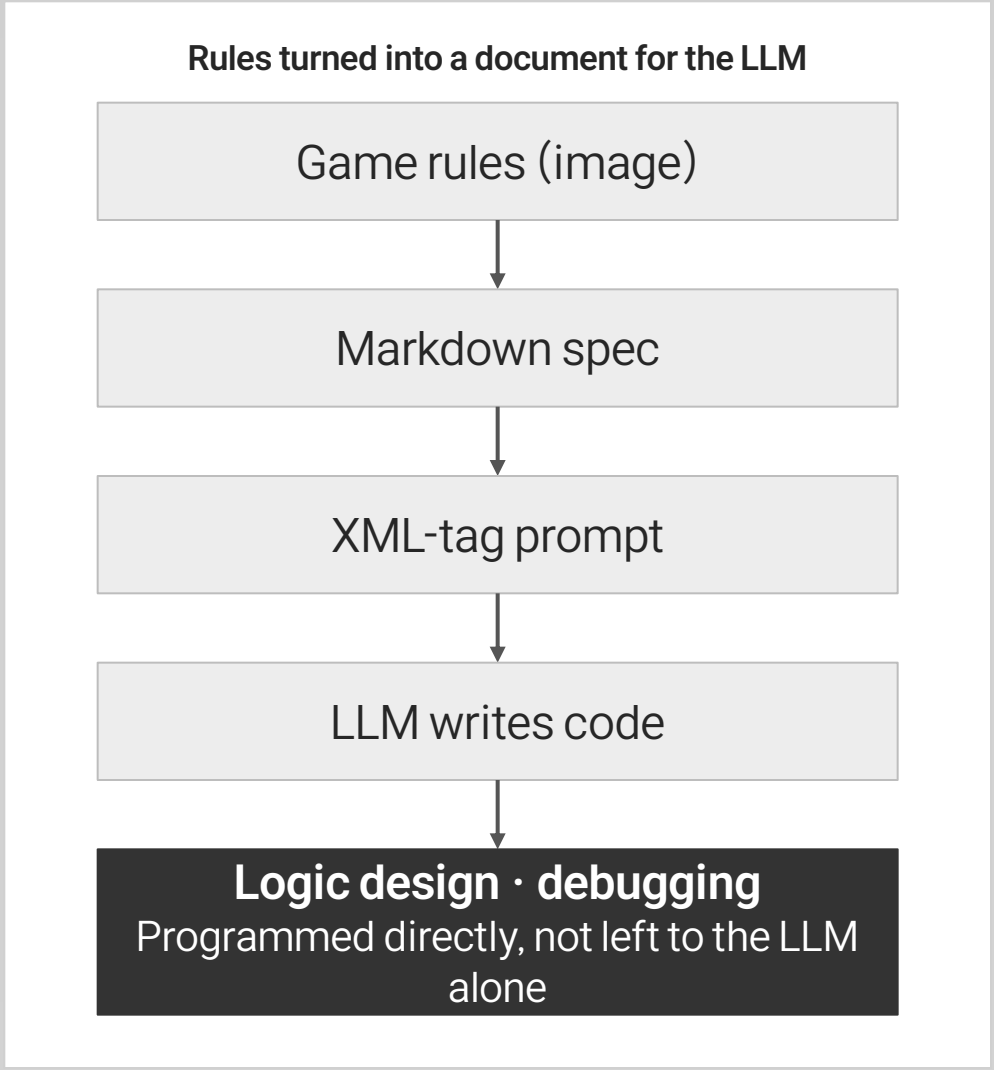
- A contest hosted by Nexon on **using LLMs**.
- In a contest where teams' **AI agents battle**, teammates built the opening, threat scoring, and fast-attack logic; I handled the **spec, code generation, and learning structure**.
- As a team of 4 we placed **37th in the qualifier** (of 1,000 teams) and won the **Best Replay Award**.



NYPC

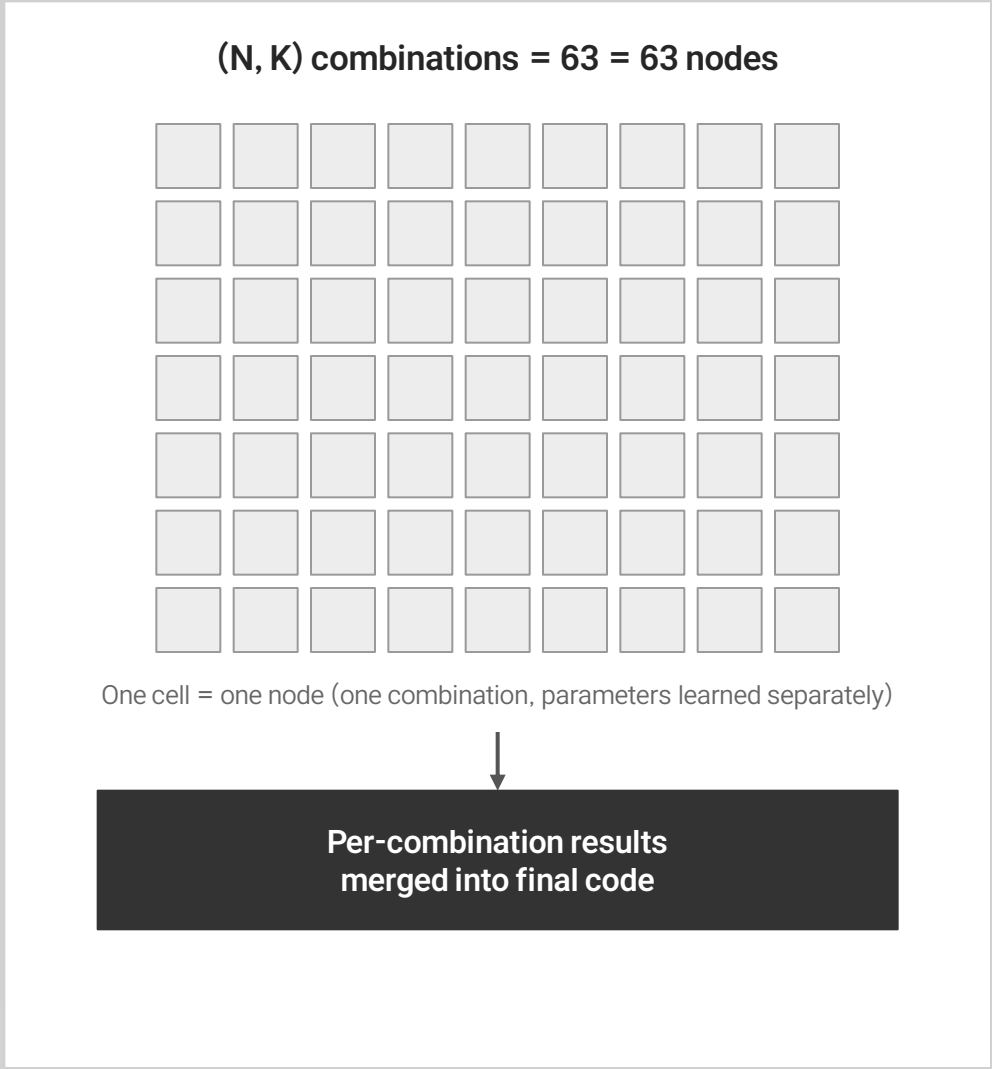
AI Usage

- Turned the rules from images into a **Markdown spec** so the LLM could understand them.
- Had the LLM write code from **XML-tag** prompts.
- I did not leave logic design and debugging to the LLM alone and **programmed them myself**.



NYPC
Learning

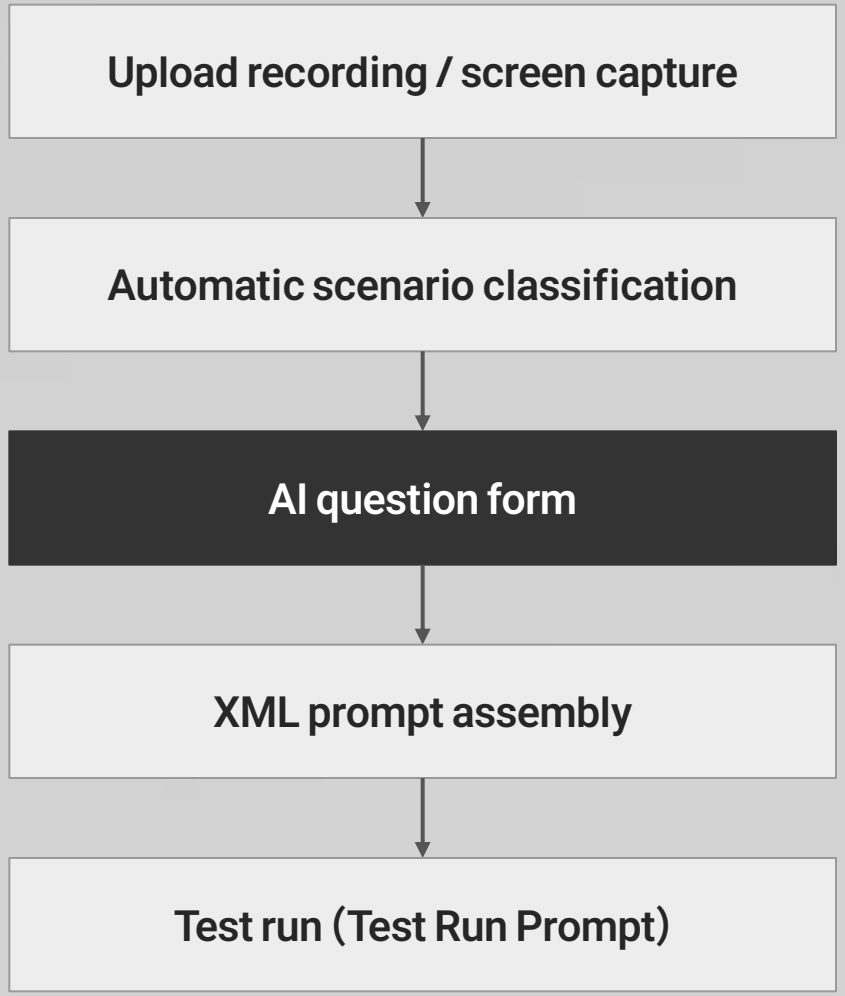
- Using AI-written code I observed the real combinations and found **63** possible nodes.
- Every (N, K) combination became a node whose parameters are **learned** separately.
- I combined this with teammates' opening, threat scoring, and fast-attack logic and submitted the final code.



TalkCompass

OVERVIEW

- An AI agent that analyzes meeting and conversation **recordings** and builds prompts tuned for Gemini.
- I handled **everything from planning to development and submission** alone.
- (Study Jam hackathon entry)



Flow from a meeting file to a finished prompt

Links

https://github.com/BarrelZebra2149/TalkCompass-Proj

TalkCompass

TalkCompass Result

- A **4-step demo** works from upload to test run.
- The AI **asks first** to fill in missing details, then writes the prompt.
- React and Spring Boot are split, with a Docker deployment setup.

React (Vite) front end

Spring Boot (Gradle) backend

Gemini API integration

Docker deployment

Submitted to the Google AI for Social Good track

MUSEMATE

OVERVIEW

- A **Flutter mobile app** that connects people through music.
- As **lead** of a team of 5, I ran the project and built the map features.
- (open-source developer contest entry)

Live chat rooms by genre

1:1 chat

Location-based music treasure hunt (map)

YouTube music streaming

Dark card = core feature I built

Links

https://github.com/BarrelZebra2149/muse_mate

MUSEMATE

MUSEMATE Result

- Built the **private music treasure hunt** and the live-room and private-board map modes.
- Managed the README, SECURITY.md, and PR template and **coordinated the merges of 5 people**.

Flutter

Firebase (chat)

Google Maps API (map)

GitHub (PR template · merges)

GridPuzzle

OVERVIEW

- Place 0–9 on an 8 × 14 grid and read **1→2→3…** as far as possible.
- I found an arrangement that reads up to **7666**.
- (reading to 8142 is a perfect score, BOJ 18789)

5	9	4	8	9	5	7	3	5	7	2	1	5	6
1	8	9	3	8	7	4	2	8	4	1	5	7	9
3	6	7	1	0	4	1	8	2	6	0	3	8	7
9	2	5	0	6	2	3	0	5	7	3	0	6	1
1	4	2	9	8	6	9	3	7	5	2	9	4	5
3	6	4	7	1	8	5	9	4	1	9	8	5	2
0	5	6	3	2	1	3	0	1	4	0	7	6	0
0	7	9	9	6	2	2	3	8	6	7	2	4	6

Arrangement found (8 × 14 grid)

Links

https://github.com/BarrelZebra2149/Project814

GridPuzzle

GridPuzzle Result

- Combined a **genetic algorithm** with **brute-force grid search**.
- Refined the mutation strategy and stagnation handling.

